2026-2 중간고사 대비

객체지향 프로그래밍 · 중간 학습 노트

단원 5개 · 레슨 23개 · 트레이너 학습 탭과 같은 내용

U1 SW 공학과 OOP 입문

L1-1 · 소프트웨어 위기 · 생명주기 · SW 공학

L1-2 · 좋은 SW의 요소 · SW 성질

L1-3 · 프로그래밍 패러다임

L1-4 · OOP 개념: ADT · 객체 · 캡슐화 · 상속 · 다형성

U2 C++ in Summary (문법 · 포인터 · 파일 입출력)

L2-1 · 컴파일 · 입출력 · string

L2-2 · 포인터 · 동적 할당 · 정적 vs 동적

L2-3 · C++ class 기초 · 객체 포인터

L2-4 · 파일 입출력

U3 객체지향 설계 (IIKH)

L3-1 · 책임 주도 설계 · 개발 과정 · 시나리오

L3-2 · 구성요소 식별 · CRC 카드 · IIKH의 6개 구성요소

L3-3 · 구성요소의 특징: 행동·상태, 결합도·응집도, 인터페이스·구현

L3-4 · 인터페이스 정형화 · 이름 · 구현 · 통합(stub) · 유지보수

U4 C++ 데이터 추상화: class

L4-1 · 데이터 추상화 · 캡슐화 · 정보 은닉

L4-2 · 클래스의 행동과 상태 · 메서드 · 메시지 전달

L4-3 · public 데이터 피하기 · getter/setter · 불변식과 assert

L4-4 · PlayingCard 예제 · 가시성 · 생성자 · 기본 생성자

L4-5 · 복사 생성자 · 얕은/깊은 복사

L4-6 · 객체 생성과 메모리 문제

L4-7 · static · const 멤버 함수 · inline · 정의와 구현 분리

L4-8 · friend · 참조

L4-9 · 인자 전달 방식 · 메모리 안전 · 지역 참조 반환

L4-10 · Copy-On-Write · TIntStack

U5 연산자 오버로딩 (Project 2 참고)

L5-1 · 연산자 함수: 멤버 방식 vs 비멤버(friend) 방식

U1 · L02 Intro to OOP.pdf + Handout 1

SW 공학과 OOP 입문

소프트웨어 위기 → SW 공학 → 좋은 SW의 요소 → 프로그래밍 패러다임 → OOP 개념. 기출 1번 빈칸(software crisis, complexity, maintenance, global/dependent)이 거의 매년 여기서 나와. 영어 단어를 정확히 외우는 게 점수야.

L1-1 · SW 공학과 OOP 입문

소프트웨어 위기 · 생명주기 · SW 공학

교안 범위: L02 p.3–6

§ 소프트웨어 위기 (Software Crisis)

- 컴퓨터 성능(power)과 문제(SW)의 **복잡도(complexity)** 가 빠르게 늘어나는데 SW 기술이 그 복잡도를 따라가지 못함
- 올바르고, 이해하기 쉽고, 검증 가능한 프로그램을 쓰기가 어려워짐
- 증상: 관리 불가(unmanageable), 유지보수 어려움, **예산 초과(over-budget)**, **일정 초과(over-time)**, 품질 낮음, 요구사항 불충족
- **원인: 복잡도 (complexity)**

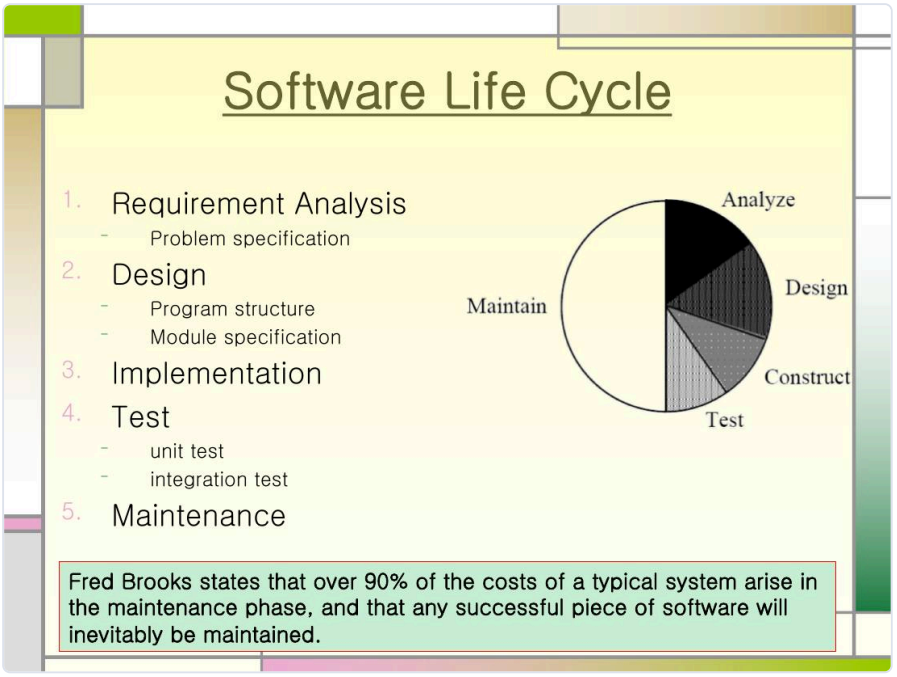
> 시험 포인트: 2025·2023·2022 빈칸. "(software crisis) arose because as computer hardware power grew rapidly, (complexity) of the software systems increased even faster".

§ 소프트웨어 생명주기 (Software Life Cycle)

1. **Requirement Analysis** (요구 분석): 문제 명세
2. **Design** (설계): 프로그램 구조, 모듈 명세
3. **Implementation** (구현)
4. **Test** (테스트): unit test, integration test
5. **Maintenance** (유지보수)

Fred Brooks: 전형적인 시스템 **비용(costs)의 90% 이상이 유지보수(maintenance) 단계**에서 생기며, 성공한 SW는 반드시 유지보수된다(will inevitably be maintained).

> 시험 포인트: 2024·2017 빈칸 "over 90% of (the costs) of a typical system arise in (the maintenance) phase, and any successful piece of software will inevitably be (maintained)".



L02 p.4 · Software Life Cycle

§ SW 공학의 정의와 목표

정의

- [IEEE] SW의 개발·운영·유지보수에 **체계적이고(systematic), 규율 있고(disciplined), 정량화 가능한(quantifiable)** 접근을 적용하는 것 = SW에 공학을 적용
- [Fritz Bauer] 신뢰할 수 있고 효율적으로 작동하는 SW를 **경제적으로(economically)** 얻기 위해 건전한 공학 원칙을 세우고 사용하는 것

목표 (Goals)

- **올바르고(correct), 효율적이고(efficient), 신뢰할 수 있는(reliable)** SW를 만든다
- SW 개발과 유지보수의 **비용(cost)을 최소화**한다

SE의 목표 4가지 (Goals of SE 슬라이드)

목표	뜻
Maintainability	변경·개선을 쉽게 할 수 있는 능력
Dependability	필요할 때 SW가 제대로 작동한다고 **믿을 수 있는** 능력
Efficiency	컴퓨팅 자원(주로 공간·시간)을 효과적으로 쓰는 능력
Usability	최종 사용자가 쉽고 효과적으로 쓰는 능력

> 시험 포인트: 2022·2019 빈칸 "(software engineering) … to (economically) obtain software that is reliable and works efficiently", 2025 "minimize (the cost)", 2017 "(Dependability) means the ability to rely on the software to function properly when needed".

L1-2 · SW 공학과 OOP 입문

좋은 SW의 요소 · SW 성질

교안 범위: L02 p.2, 7 + Handout 1

§ 좋은 소프트웨어의 요소 (Handout 1)

요소	뜻
correct (correctness)	사용자 요구·명세를 오류 없이 만족
efficient (efficiency)	자원(공간·시간, HW)을 최적으로 써서 빠르게 응답
easy to maintain (maintainability)	읽기 쉽고(readability), 고치기 쉽고(modifiability), 확장하기 쉽고(extensibility, scalability), 재사용하기 쉽고(reusability), 디버그·테스트 쉽고(testability), 모듈화(modularity), 이식 쉬움(portability)
stable, reliable, robust	안정성, 신뢰성, 견고성
easy to use (usability)	쓰기 쉬움
secure (security), integrity	무단 접근·손상·악의적 공격으로부터 데이터 보호
interoperability	표준 프로토콜·인터페이스로 다른 시스템과 데이터 교환
concise, complete, consistent, well-documented, well structured	간결, 기능 완비, 일관성(명명 규칙 등), 문서화, 구조화

시험 포인트: maintainability에 들어가는 "easy to read, modify, extend, reuse, debug/test, port"를 영어로 쓸 수 있어야 해 (2019·2016 서술 "A program with high maintainability is easy to (read), easy to (modify), easy to (extend)...").

§ 안정성 · 신뢰성 · 견고성

- **stability**: 조건이 바뀌어도 시간이 지나도 올바른 기능을 유지 (어떨 땐 되고 어떨 땐 안 되면 unstable)
- **reliability**: 정해진 조건에서 정해진 기간 동안 요구 기능을 **실패 없이 일관되게** 수행하는 능력
- **robustness**: 예외 상황(잘못된 입력, 재해)에서도 기능을 유지
- 세 단어는 뜻이 거의 같고 강조점만 조금 다르다

함정: 2025 빈칸 "(reliability) means the ability of the software to consistently perform its required functions without failure under specified conditions for a specified period" vs 2017 "(Dependability) means the ability to rely on the software to function properly when needed". 정의 문장으로 구별해.

§ SW 성질 (Software Properties)

- **Modularity** – divide and conquer (분할 정복)
- **Encapsulation** – hide the implementation (구현 숨기기)
- **Localization** – collect similar things together (비슷한 것 모으기)
- **Abstraction** – simplifying complex reality (복잡한 현실 단순화)
- **Uniformity** – make everything look similar
- **Completeness** – do everything required
- **Confirmability** – be able to prove that the software works properly

프로그래밍 패러다임

교안 범위: L02 p.8–15

§ 비구조적 프로그래밍 (Unstructured Programming)

- **main 프로그램 하나**로만 구성, 명령어들의 나열
- **전역 데이터만** 사용
- **goto 문**을 사용
- 단점: 같은 문장 시퀀스가 반복되면 다 다시 써야 함
- **스파게티 코드**: goto가 여기저기 점프해서 구조가 복잡하게 얽힘 → 이해·흐름 추적·수정이 거의 불가능

> 시험 포인트: 2024 빈칸 "(Unstructured programming) makes extensive use of GOTO statements... a program contains only a single code block".

§ 절차적 · 모듈 프로그래밍

절차적 프로그래밍 (Procedural) — 스파게티 코드의 남용 때문에 구조적 언어가 등장

- 프로그램 = **프로시저 호출의 나열**, 프로시저 = 같은 문장 시퀀스를 묶은 것
- 반복이 줄고 오류가 줄어듦
- **구조적 프로그래밍 (Structured Programming) ≒ top-down approach ≒ divide and conquer ≒ functional decomposition**

모듈 프로그래밍 (Modular)

- SW가 분리된 부분(**module**)들로 구성. 공통 기능의 프로시저들을 모듈로 묶음
- 프로그램은 **인터페이스**로 모듈을 사용: **interface는 보이고(visible), implementation은 숨겨짐(hidden)**
- 유지보수성 향상

> 시험 포인트: 2022 빈칸 "structured programming ≒ (top-down approach) ≒ (divide and conquer) ≒ functional decomposition".

§ 전역 변수를 쓰지 마라

- **전역 변수 사용은 위험(unsafe)**
  - 데이터(전역 변수) 공유는 모듈 프로그래밍 원칙 위반
  - 모든 모듈이 아무 제약 없이 모든 전역 변수에 접근 → 어떤 모듈도 독립적으로 개발·이해할 수 없음
  - **모든 모듈이 서로 의존(dependent on one another)** 하게 만든다
- 해결: **Modularized Data** — 데이터를 모듈 안에 **지역화(localize)** → 모듈끼리 더 독립적 → 안전

> 시험 포인트: 2024·2022·2019·2017·2016 거의 매년! "Using (global) variables is often unsafe because this makes all modules (dependent) on one another."

§ 절차적/모듈 방식의 문제와 OOP

절차적·모듈 방식의 문제

- 데이터와 연산이 **분리(decoupled)** 되어 있음
- 큰 시스템에서 잘 안 됨: 디버그·확장·유지보수 어려움, 코드 재사용 제한
- 명시적인 생성·소멸

OOP

- 프로그램 = **상호작용하는 객체들의 집합**, 각 객체가 자기 상태를 유지
- **데이터와 연산을 객체 하나로 결합**
- 구조적 = "data + function = program", OOP = "data + function = object, **object + object = program**"
- 문제 영역을 처리의 흐름이 아니라, 서로 **메시지를 주고받는 객체들**로 본다
- 객체는 SW의 **부품(component)** 이고, 객체의 **인터페이스는 잘 정의**되어야 함

Why OO? 큰 SW 개발에 좋음, 유지보수성, **재사용(reuse, not rewrite)** → 생산성·품질·이해도·확장성·수명 향상. 이미 잘 테스트된 재사용 모듈을 쓰므로 **신뢰성도 증가**.

> 시험 포인트: 2024 빈칸 "(Object-oriented programming) is a programming paradigm based on the concept of (objects), which can contain data and code".

L1-4 · SW 공학과 OOP 입문

OOP 개념: ADT · 객체 · 캡슐화 · 상속 · 다형성

교안 범위: L02 p.24–33

§ 추상 자료형 (ADT)

- **ADT**: 데이터 변수들과 관련 연산들의 집합으로, **특정 구현과 독립적으로 정밀하게 명세된** 것
- **ADT = data + operations (no implementation)**
- OOP에서는 프로시저를 정의하려고 데이터를 정의하는 게 아니라, **데이터를 정의하는 데 필요한 프로시저를 정의**한다
- ADT는 데이터와 프로시저를 결합한 타입. C++에서는 **class**로 ADT를 정의
- 사용자는 ADT가 **무엇을 하는지(what)** 만 알면 되고 **어떻게 하는지(how)** 는 몰라도 됨 (예: stack의 push/pop)

> 시험 포인트: 2025 서술 "ADT is called abstract because … (its data and operations are specified independently of any particular implementation)".

§ 클래스와 객체

- **class**: 객체의 속성(attributes = **member variables**)과 행동(behaviors = **member functions**)을 정의
- **object**: **클래스의 인스턴스(instance of a class)**. 내부 데이터, 내부 연산, **인터페이스 연산**으로 구성
  - identity, attribute, behavior(method), 다른 객체와의 관계를 가짐
  - **클래스의 메서드만** 객체의 내부 데이터에 접근 가능
  - **메시지 전달 (message passing)**: 메서드를 호출해서 다른 객체에 작업을 요청
- **Class vs Object**
  - 같은 클래스로 만든 객체들은 서로 **다른 객체** (각자 다른 메모리 공간·주소)
  - 같은 클래스의 객체들은 클래스의 **메서드를 공유**
  - 객체는 실행 시간에 동적으로 생성·소멸될 수 있다

> 시험 포인트: 2016 빈칸 "(An object) is an instance of a class".

§ 정보 은닉 · 캡슐화 (개요)

- **캡슐화**는 메시지를 보내는 객체들로부터 클래스의 기능적 세부를 **감춘다 (conceals the functional details)**
- 객체의 **내부 데이터는 인터페이스를 통해서만 수정**될 수 있다
- 객체 그림: 바깥에 interface operations(specification), 안에 data와 internal operations(implementation)

> 시험 포인트: 2025 빈칸 "In the concept of information hiding, (internal data) of an object can be modified only through (an interface)". 자세한 건 U4.

§ 상속과 다형성

- **상속 (Inheritance)**: 자식 클래스(sub class)가 부모 클래스(super class)의 속성·행동을 물려받고 자기 것을 추가. 자식의 속성은 부모와 대부분 비슷해야 함. **목표: 객체(클래스) 재사용** → 대형 SW 개발 비용 절감
- **다형성 (Polymorphism)**: 같은 메서드(메시지)가 다른 의미를 가질 수 있음. 서로 다른 타입의 객체가 같은 이름의 메서드 호출에 각자 타입에 맞게 반응
  - C++에서: **virtual function, operator overloading, function overloading, template**

- 동적 바인딩 (dynamic binding)
- **객체 모델의 이점**: 시간이 지나며 진화하는 시스템(확장 가능, 안정적), 객체·클래스로 생각하는 게 사람에게 쉬움, 클라이언트와 구현자를 분리해 캡슐화로 실수로 인한 손상 방지, 재사용

함정: 상속·virtual은 중간 이후 진도(교안 일정 6–7주차지만 올해는 아직). 대시보드의 '범위 밖' 표시와 같음.

U2 · C++ in Summary.pdf + static vs dynamic 토론 자료

C++ in Summary (문법 · 포인터 · 파일 입출력)

C++ 문법 요약. 시험에 직접 나오는 건 포인터 *·&의 세 가지 뜻, 동적 할당과 정적 할당 비교, Rectangle 포인터 코드 빈칸, 그리고 파일 입출력(open mode, tellg/seekg, 바이너리 파일 복사)이야. 파일 복사 코드는 2024·2025 연속으로 통째로 썼어.

컴파일 · 입출력 · string

교안 범위: C++ in Summary p.2–14

§ 컴파일과 표준 입출력

- **컴파일러**: 고급 언어 코드 → 기계어. C++ 컴파일러는 소스 파일에서 실행 파일(binary)을 만든다
  - `g++ -o TestProg TestProg.cpp` → `./TestProg`
- `#include <iostream>` (header file), `using namespace std;`. C++ 프로그램에서 C 코드를 써도 된다
- **cout**: 화면 스트림 객체 (standard output). `cout << "Hello";` → 문자열 Hello 출력, `cout << Hello;` → 변수 Hello의 값 출력
- **cin**: 키보드 스트림 객체 (standard input). `cin >> age;` → 키보드 입력이 변수로 (화살표 방향이 데이터 흐름)
- 서식: printf와 비슷하게 `std::format` (C++20), `std::print` (C++23)

```
double pi = 3.141592; int age = 21;
printf("pi=%7.2f , age=%d\n", pi, age);
cout << format("pi={:7.2f} , age={}\n", pi, age);   // pi=   3.14 , age=21
```

§ 자료형 · string · 제어문 · 함수 · 배열

- 변수 = 값이 저장되는 **이름 붙은 메모리 위치**, 사용 전에 선언
- char 1B, short 2B, int 4B (-2^{31} ~ $2^{31}-1$), float 4B, double 8B. 양수만이면 unsigned
- **string**: 표준 라이브러리 클래스. C 문자열(char*)과 달리 **메모리를 자동 관리**하고 편리한 함수 제공 (length, find, substr, replace, insert, erase, c_str, stoi, stod, to_string)
- `getline(cin, str)`: 한 줄 읽기
- for문 실행 순서: (1) 초기화 → (2) 조건 → body → (3) 증감 → (2) → body → ...
- break = 루프 탈출, continue = 다음 반복의 조건 검사로
- 함수: 지역 변수(매개변수 포함)는 호출 때 생성, 반환 때 삭제. 쓰는 이유 = 반복 제거, 재사용, top-down, 디버그 쉬움, 수정·확장 쉬움, 가독성
- 배열: 같은 타입 원소의 **고정 크기** 모음, 메모리에 연속 저장, 빠른 임의 접근 `x[15]=3;`

포인터 · 동적 할당 · 정적 vs 동적

교안 범위: C++ in Summary p.23–24 + 토론 자료

§ 포인터

- 포인터 = 메모리 **주소**를 저장하는 변수. 크기는 항상 8바이트(64비트)/4바이트(32비트)

```
int *xPtr, x;   // xPtr은 포인터, x는 int
x = 5;
xPtr = &x;      // xPtr이 x를 가리킴 (xPtr의 값 = x의 주소)
*xPtr = 10;     // xPtr이 가리키는 값을 10으로 → x가 10
cout << x;      // 10
```

`*`**의 세 가지 뜻**: (i) 곱셈 (이항) `z=x*y;` (ii) 가리키는 값 (단항, the-value-of) `*x=5;` (iii) 포인터 타입 선언 `int *x;`
`&`**의 세 가지 뜻**: (i) 비트 AND (이항) `z=x&y;` (ii) 주소 연산자 (단항, address-of) `x=&y;` (iii) 참조 타입 선언 `int& y=x;`

> 시험 포인트: 2022 기말 "Meaning of '*' in line 2? '&' in line 3?" 같은 유형. 줄마다 어느 뜻인지 영어로 답할 수 있어야 해 (pointer type declaration / reference type declaration / address-of / dereference / multiplication / bitwise AND).

§ 동적 메모리 할당

- "dynamic" = **실행 시간(run-time, execution-time)에 일어난다**
- C의 `malloc` ⇔ C++의 `new`, C의 `free` ⇔ C++의 `delete`

```
int *x = new int[10];   // C: x = (int*)malloc(10*sizeof(int));
...
delete[] x;             // C: free(x);
```

- 객체 하나는 `delete p;`, 배열은 `delete[] p;`
- **Java와 달리 C++은 직접 해제**해야 함 (garbage collection 없음)

> 시험 포인트: 2019 빈칸 "Dynamic memory allocation means memory allocation occurs at (run-time)".

§ 정적 할당 vs 동적 할당 (토론 자료)

`int x[10];` (정적) vs `int* y = new int[n];` (동적)

	장점	단점
정적	간단하고 빠름, 메모리가 **자동으로 해제**됨 (스택, 범위를 벗어나면)	크기가 **컴파일 시간**에 고정 → 실행 중 바꿀 수 없음 (낭비 또는 부족)
동적	크기를 **실행 시간**에 정할 수 있음 → 메모리를 필요한 만큼 효율적으로	프로그래머가 **직접 해제(delete[])** 해야 함 → **memory leak, dangling pointer** 위험

> 시험 포인트: 2023 서술이 이 토론 자료 그대로 나옴 (장점·단점·이유 4칸). 핵심 단어 = **compile time vs run time**, **automatic vs manual deallocation**.

C++ class 기초 · 객체 포인터

교안 범위: C++ in Summary p.25–28

§ class · 생성자 · 소멸자 (요약)

- **class**: 멤버 데이터와 멤버 함수를 담는 자료구조 타입. 새 데이터 타입을 정의
- **생성자/소멸자**: 객체가 **생성/삭제될 때 자동으로 호출**되는 멤버 함수
  - 지역 객체는 함수가 반환될 때 삭제 → 그때 소멸자 호출. 소멸자 이름은 `~클래스이름`
  - 생성자를 안 만들면 컴파일러가 빈 기본 생성자를 만듦. 소멸자도 마찬가지
  - 멤버 변수를 초기화하지 않으면 **쓰레기 값(garbage)** → 위험·불안정. 생성자에서 초기화해야 함

§ 객체 포인터 (Rectangle 예제)

```
class Rectangle {
  private:
    int width, height;
  public:
    Rectangle() { width=1; height=1; }
    Rectangle(int x, int y) : width(x), height(y) {}
    int area() { return width*height; }
};
int main() {
  Rectangle obj(3,4);
  Rectangle *foo, *bar, *baz;
  foo = &obj;                     // 주소 대입
  bar = new Rectangle(5,6);       // 동적 생성: new는 주소를 반환
  baz = new Rectangle[2];         // 배열: 기본 생성자 2번 → 각 1×1
  cout << "obj's area: "    << obj.area()    << '\n';   // 12
  cout << "*foo's area: "   << foo→area()   << '\n';   // 12
  cout << "*bar's area: "   << bar→area()   << '\n';   // 30
  cout << "baz[0]'s area: " << baz[0].area() << '\n';   // 1
  cout << "baz[1]'s area: " << baz[1].area() << '\n';   // 1
  delete bar;
  delete[] baz;
  return 0;
}
```

- **객체는 `.`, 포인터는 `->`** (= `(*foo).area()`). 배열 원소 `baz[0]`은 객체라서 `.`
- `foo = obj;` (X) → `foo = &obj;` (O). `bar.area()` (X) → `bar→area()` (O)

> 시험 포인트: 2025·2024·2023·2019·2017 매년 이 코드의 변형 (빈칸 채우기, 오류 5개 찾기, 출력 쓰기). `.` / `->` / `&` / `new` / `delete[]`를 줄 단위로 정확히.

파일 입출력

교안 범위: C++ in Summary p.29–39

§ 파일 스트림 클래스와 open

- **ofstream**: 파일에 쓰기 / **ifstream**: 파일에서 읽기 / **fstream**: 읽기·쓰기 둘 다 (`#include <fstream>`)
- 파일 처리 순서: **open → read/write → close**
- 열 때 정할 것: 텍스트 vs 바이너리, 쓰기라면 **추가(append) vs 덮어쓰기(truncate)**, 파일 위치(자동 vs 사용자 제어)
- `myfile.open(filename, mode);` 또는 생성자 `ofstream myfile("example.txt");`
- 열렸는지 확인 `if (myfile.is_open()) { ... }`, 닫기 `myfile.close();`
- 텍스트 쓰기 `myfile << "This is a line.\n";`, 읽기 `while (getline(myfile, line)) cout << line << '\n';`
- 상태 검사: `bad()` 읽기/쓰기 실패, `fail()` bad + 형식 오류, `eof()` 파일 끝, `good()` 다 괜찮음, `clear()` 플래그 초기화

> 시험 포인트: 2019 빈칸 "ofstream: write on files / (ifstream): read from files / (fstream): read and write".

§ 파일 열기 모드

플래그	뜻
`ios::in`	입력(읽기)용으로 열기
`ios::out`	출력(쓰기)용으로 열기
`ios::binary`	바이너리 모드
`ios::ate`	초기 위치를 **파일 끝**으로 (안 쓰면 처음)
`ios::app`	**모든 출력을 파일 끝에** → 기존 내용에 **덧붙임(append)**
`ios::trunc`	출력용으로 열 때 파일이 **이미 있으면** 기존 내용을 **지우고** 새 내용으로

- 여러 개는 **비트 OR** `|` 로 결합: `myfile.open("example.bin", ios::out | ios::app | ios::binary);`
- **기본 모드**: ofstream = `ios::out`, ifstream = `ios::in`, fstream = `ios::in | ios::out`
- 쓰기로 열면 기존 파일이 지워지고 처음부터 써짐 (교수님이 여러 번 경고)

> 시험 포인트: 2023 빈칸 "ios::app means all output operations are performed at the (end)", "(ios::trunc) means if the file is opened for output and already (existing), its previous content is deleted". 2019도 app.

§ 파일 위치: tellg / seekg

- **get 위치**(읽기)와 **put 위치**(쓰기)가 따로 있다
- `tellg()` / `tellp()`: 현재 get/put 위치 반환
- `seekg(position)` / `seekp(position)`: 위치를 절대값으로 이동
- `seekg(offset, direction)`: `ios::beg` (처음부터), `ios::cur` (현재 위치부터), `ios::end` (끝부터)

파일 크기 구하기 (매년 빈칸)

```
streampos begin, end;
ifstream myfile("example.bin", ios::binary);
begin = myfile.tellg();          // 처음 위치
myfile.seekg(0, ios::end);       // 끝으로
end = myfile.tellg();            // 끝 위치
myfile.close();
cout << "size is: " << (end-begin) << " bytes.\n";
```

시험 포인트: 2024·2023·2019 "Fill in the boxes to print the size". `begin = myfile.tellg();` → `myfile.seekg(0, ios::end);` → `end = myfile.tellg();` 세 줄.

---

§ 바이너리 파일과 파일 복사

- 텍스트 vs 바이너리: 253을 텍스트로 쓰면 문자 '2','5','3' (3바이트), 바이너리로 쓰면 11111101 (1바이트)
- 바이너리 입출력은 `write(memory_block, size);` / `read(memory_block, size);` (memory_block은 `char*`)

파일 전체 읽기 (교안)

```
ifstream file("example.bin", ios::in | ios::binary | ios::ate);
if (file.is_open()) {
    streampos size = file.tellg();     // ate라서 끝 위치 = 파일 크기
    char* memblock = new char[size];
    file.seekg(0, ios::beg);           // 처음으로 되감기
    file.read(memblock, size);
    file.close();
    ...
    delete[] memblock;
}
```

파일 복사 (2024·2025 코드 작성): 위 코드 + `ofstream out("des.bin", ios::out | ios::binary); out.write(memblock, size); out.close();`

주의: getline으로 한 줄씩 복사하면 바이너리 파일이 깨지고 감점 (기출 해설). `ios::binary`, `ios::ate` → `tellg` → `seekg(0, ios::beg)` → `read` → `write` → `delete[]` 순서를 외워.

U3 · L03 Object-Oriented Design.pdf (Budd Ch.3)

객체지향 설계 (IIKH)

책임 주도 설계와 개발 9단계, IIKH 예제, 구성요소의 특징(결합도·응집도, 인터페이스·구현), 통합과 stub, 유지보수. 결합도/응집도 빈칸은 2024·2023·2022·2019·2016에 다 나왔고, stub·slowly integrated 서술도 단골이야.

책임 주도 설계 · 개발 과정 · 시나리오

교안 범위: L03 p.2–9

§ 책임 주도 설계 (Responsibility Driven Design)

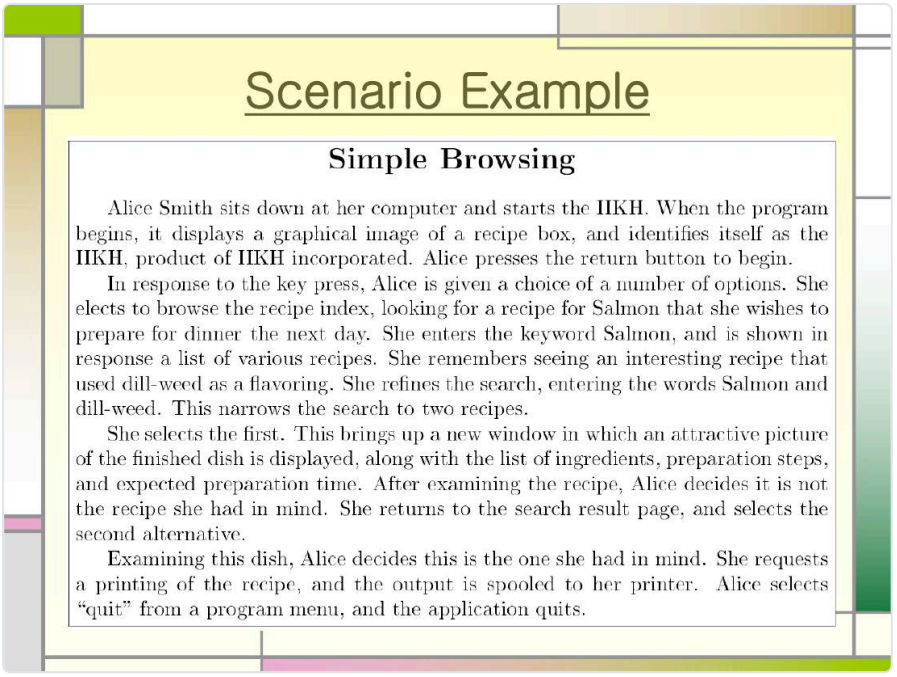
- OOP = 상호작용하는 객체들의 집합
- 각 객체는 공동체(community) 안에서 **책임(responsibility)** 을 가진 구성요소
- 시스템이 수행할 활동들을 나누고, 그 활동(책임)을 시스템의 객체들에 **분배**
- 책임 목록을 다 만든 뒤, 객체 집합을 정하고 어떤 책임을 어떤 객체에 줄지 결정
- **한 객체 안의 책임들은 서로 밀접하게 관련**되어야 함
- 설계 과정: 부정확하고 모호하고 불분명한 **명세(specification)** → 구체적 논리의 **SW 시스템 설계**

§ 개발 과정 9단계

1. **Initial Description** (Specification) — 초기 명세
2. **Refine Specification** — 명세 다듬기 (시나리오)
3. **Identification of Components** — 구성요소 식별
4. **Identification of Components Responsibilities** — 구성요소의 책임 식별 (CRC 카드)
5. **Formalize the Interface** — 인터페이스 정형화
6. **Designing the Subsystem** — 설계
7. **Implementing Components** — 구성요소 구현 (+ unit test)
8. **Integration of Components** — 통합 (stub)
9. **Maintenance & Evolution** — 유지보수와 진화

§ IIKH와 시나리오

- **IIKH (Intelligent Interactive Kitchen Helper)**: 부엌의 레시피 색인 카드 상자를 대체하는 PC 응용. 레시피 DB 관리 + 일주일 같은 기간의 식사 계획, 인원수에 맞게 레시피 자동 조절(scale), 주간·일·끼니별 메뉴 출력, 통합 장보기 목록 출력
- 초기 명세는 **모호하고 불분명** (사장이 냅킨 뒤에 그려 옴) → 모호함을 없애야 함
- **명세 다듬기 = 시나리오 따라가기 (walking through scenarios)**: 이미 동작하는 응용이 있다고 **가정하고** 다양한 사용 방법을 따라가 본다
  - look and feel 정하기, 의도한 용도를 빠짐없이 찾기, 설명 문서와 상위 설계 만들기
  - 다른 저자들은 **use-cases**라고 부름 (시스템 사용 시나리오의 집합)
- IIKH 기능: 레시피 DB 탐색, 새 레시피 추가, 레시피 편집·주석, 여러 코스의 식사 계획, 인원수에 맞게 조절, 기간(일주일) 계획, 장보기 목록 생성



L03 p.8 · IIKH 시나리오 예

구성요소 식별 · CRC 카드 · IIKH의 6개 구성요소

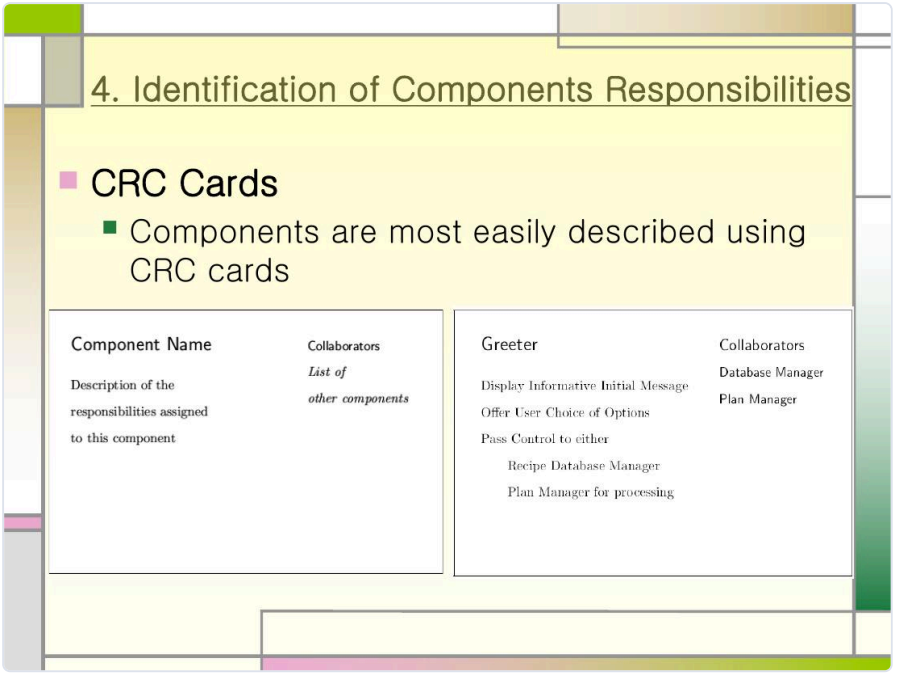
교안 범위: L03 p.10–18

§ 구성요소 (Component)

- **추상적 설계 단위**로, 서로 다른 작업에 대한 책임을 연결할 수 있는 것
- **작고 잘 정의된 책임 집합**을 가져야 함
- 다른 구성요소와의 상호작용은 **가능한 최소**로

§ CRC 카드

- 구성요소는 **CRC 카드**로 가장 쉽게 기술한다: **C**omponent(이름), **R**esponsibility(맡은 책임 설명), **C**ollaborators(협력하는 다른 구성요소 목록)
- 예) **Greeter**: 책임 = 환영 메시지 표시, 사용자에게 선택지 제공, Recipe Database Manager나 Plan Manager로 제어 넘기기 / 협력자 = Database Manager, Plan Manager



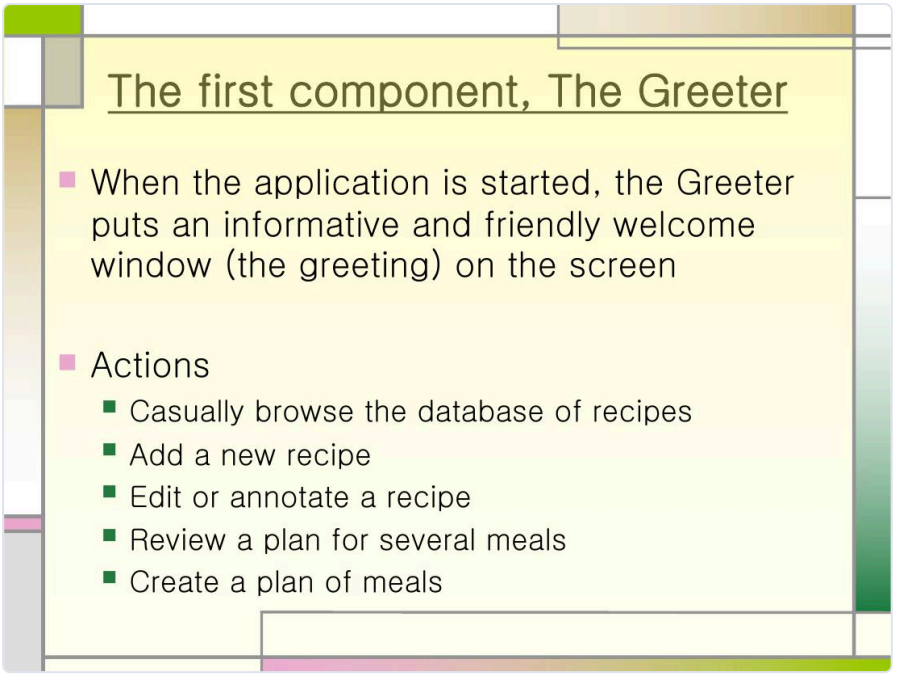
L03 p.11 · CRC 카드 양식과 Greeter 카드

§ IIKH의 6개 구성요소

구성요소	책임
Greeter	시작 시 환영 창. 레시피 DB 탐색, 새 레시피 추가, 레시피 편집·주석, 식사 계획 검토·생성 선택
Recipe Database	레시피 DB 유지, 탐색 허용, 기존 레시피 편집·주석, 새 레시피 추가
Recipe	재료 목록과 변환 알고리즘 유지, 자기 데이터 편집, 화면에 자기 표시, 자기 인쇄
Planner (Plan Manager)	계획할 날짜 범위 선택, 기존 계획 편집, Date 객체와 연관
Date	특정 끼니 편집, 날짜에 주석("Bob's Birthday"), 전체 끼니의 장보기 목록 출력
Meal	한 끼 정보, 레시피 DB와 상호작용해 레시피 선택, 인원수 설정 → 레시피 자동 조절, 끼니 장보기 목록

- 6개 구성요소만으로 다 됨 → 이 시점에 구성요소를 **다른 프로그래머들에게 나눠** 개발시킬 수 있다

> 시험 포인트: 2025 서술 "In project 1 (IIKH), main difference between Recipe class and RecipeDB class?" → Recipe = 레시피 **하나**의 데이터를 유지하고 자기를 편집·표시·인쇄, RecipeDB = 레시피 **모음**을 관리(탐색, 검색, 추가, 편집).



L03 p.12 · 첫 구성요소 Greeter

구성요소의 특징: 행동·상태, 결합도·응집도, 인터페이스·구현

교안 범위: L03 p.19–25

§ 행동과 상태 · 인스턴스와 클래스

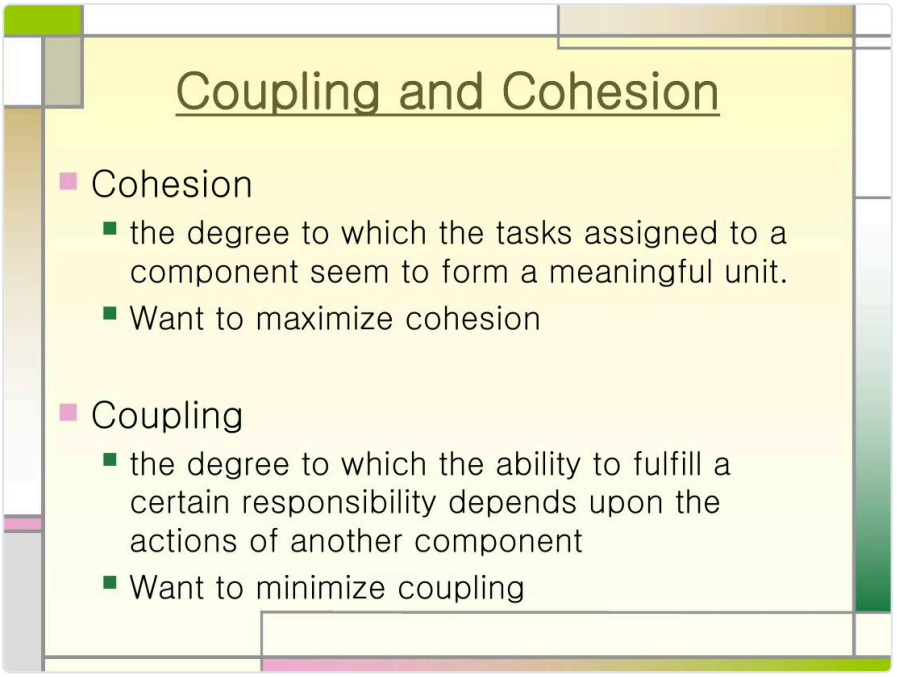
- **Behavior**: 구성요소가 수행할 수 있는 행동들의 집합
- **State**: 구성요소 안에 있는 모든 정보(데이터 값). 행동이 상태를 바꾸는 게 일반적
- **Instances and Classes**: Recipe 인스턴스는 많지만 모두 같은 방식으로 행동 → 그 행동은 **Recipe 클래스에 공통**

§ 결합도와 응집도 (Coupling and Cohesion)

	정의	목표
Cohesion (응집도)	한 구성요소에 배정된 작업들이 **의미 있는 하나의 단위**를 이루는 정도 (책임들이 얼마나 강하게 관련되어 있나)	**최대화 (maximize)**
Coupling (결합도)	어떤 책임을 수행하는 능력이 **다른 구성요소의 행동에 의존하는** 정도 (모듈 간 상호 의존도)	**최소화 (minimize)**

- 좋은 설계 = **high cohesion, low (loose) coupling**
- 결합도가 높으면: 한 모듈의 변경이 다른 모듈에 영향 → 유지보수·재사용이 어려움

> 시험 포인트: 2024 "(Coupling) and (Cohesion) are two key concepts used to measure design quality... good design = (high) cohesion, (low) coupling", 2023 "Coupling is the degree to which each module (relies/depends) on ... (Cohesion) ... (minimize) ... (maximize)", 2016 "(depends upon) ... desirable to (minimize)". 영어 정의 문장을 그대로 외워.

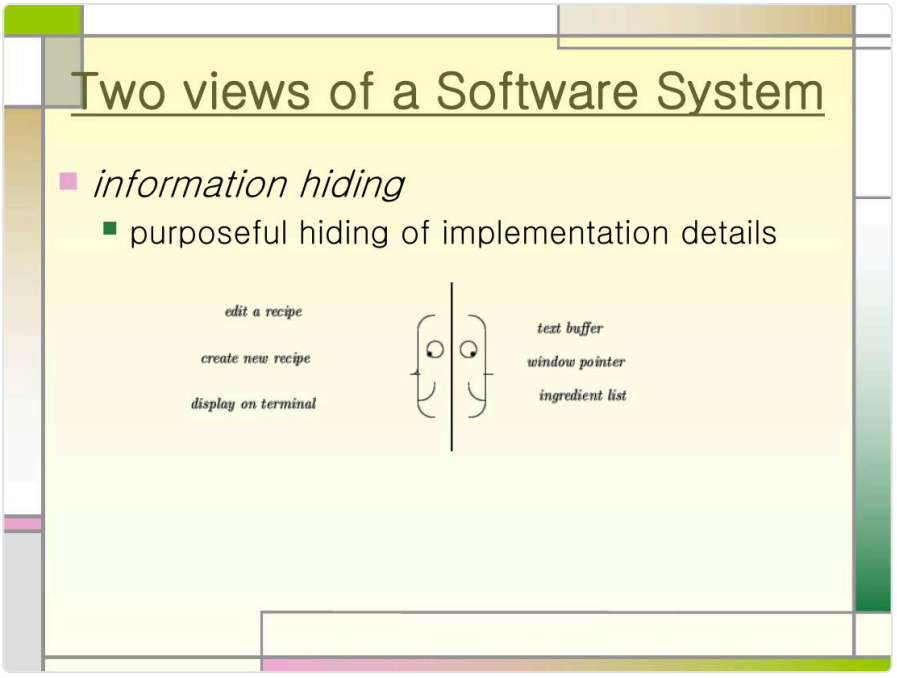


L03 p.22 · Coupling and Cohesion

§ 인터페이스와 구현 · 정보 은닉

- 구성요소를 **무엇을 할 수 있는지(what)** 로 특징지음

- 구성요소의 사용자는 **무엇을 하는지만 알면 되고 어떻게 하는지(how)는 몰라도 됨**
- **Two views of a software system**: 인터페이스(edit a recipe, create new recipe, display on terminal) vs 구현(text buffer, window pointer, ingredient list)
- **information hiding**: 구현 세부를 **의도적으로(purposeful) 숨기는 것**. 덮개(cover)가 있어서 내부를 볼 수 없고, 덕분에 **보호(protection)** 도 됨



L03 p.24 · Two views of a Software System

§ Public view와 Private view

- **Public view**: 다른 구성요소가 보고 쓸 수 있는 특징(데이터 또는 행동) → 인터페이스
- **Private view**: 구성요소 **안에서만** 쓰는 특징 → 구현
- public/private을 구분하는 것이 객체지향 프로그래밍의 핵심 특징 (교수님 강조)

인터페이스 정형화 · 이름 · 구현 · 통합(stub) · 유지보수

교안 범위: L03 p.26–33

§ 인터페이스 정형화와 이름 짓기

5. Formalize the Interface: 구성요소 간 통신 채널을 정형화

- 각 구성요소의 일반 구조 식별. 행동이 **하나뿐인 구성요소는 함수**로, **여러 개면 클래스**로
- 책임마다 이름을 붙임 → 나중에 프로시저 이름이 됨. 함수 매개변수 식별, 정보를 구성요소에 배정, 시나리오를 다시 돌려 필요한 데이터가 다 있는지 확인

이름 선택 지침

- **evocative** (문제 맥락에서 의미를 떠올리게 하는) — 예: 의미 없는 이름보다 무엇을 세는지 드러나는 이름
- **short** (짧게)
- **pronounceable** (발음할 수 있게, 소리 내어 읽어 보기)
- **consistent** (프로젝트 안에서 일관되게, 예: 변수는 소문자로 시작·함수는 대문자로 시작 규칙을 정했으면 끝까지)
- **avoid digits** (이름에 숫자 피하기: count1, count2는 차이를 알 수 없음)

> 시험 포인트: 2016 서술 "What does 'evocative' mean?" → The name should call to mind (suggest) the meaning/role of the thing in the problem domain.

§ 설계 · 구현 · 단위 테스트

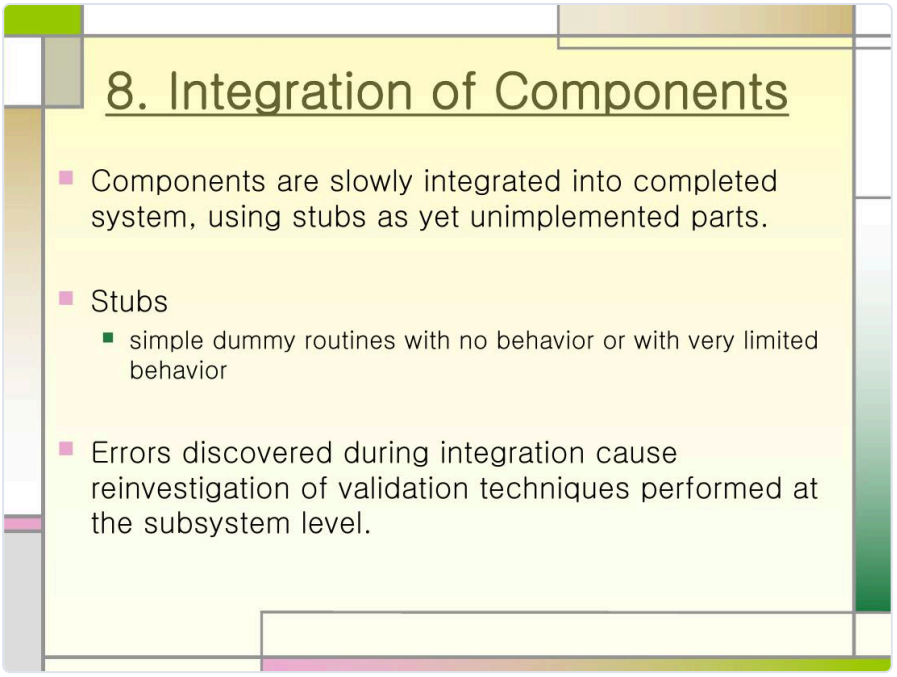
- **6. Design**: 설명을 SW 배치로 바꿈, 각 구성요소의 자료구조 설계, 행동 설명을 알고리즘으로
- **7. Implementing Components**: 각 구성요소의 활동 구현. 팀원에게 구성요소를 나눠 맡김
  - **Unit testing**: 서브시스템을 **개별적으로 검증(validate)**. 올바르게 동작하기 위한 조건을 찾고, 합법적인 입력 값으로 테스트

§ 통합과 stub

8. Integration of Components: 구성요소들을 **천천히(slowly) 통합**해 완성 시스템으로, 아직 구현 안 된 부분은 **stub**으로

- **stub**: 행동이 없거나 아주 제한된 행동만 하는 **간단한 더미 루틴 (simple dummy routine)**. 아직 개발 안 된 구성요소의 임시 대체물
- **slowly integrated**: 한꺼번에(all at once) 통합하지 않고 **하나(또는 몇 개)씩 추가하며 테스트** → 문제가 생기면 방금 추가한 부분이 원인이라 찾고 고치기 쉬움. 한꺼번에 하면 어디서 생긴 문제인지 알 수 없음
- 통합 중 발견된 오류는 서브시스템 수준의 검증 기법을 다시 조사하게 만든다

> 시험 포인트: 2025·2016 서술 "Meaning of 'slowly integrated'? Meaning of 'stubs'?", 2022 빈칸 "(Stub) is a simple dummy routine with no behavior or with very limited behavior".



L03 p.30 · 8. Integration of Components (stub)

§ 유지보수와 진화 · 흔한 설계 결함

9. Maintenance and Evolution

- **Software maintenance**: 초기 동작 버전을 **전달한 뒤에(subsequent to the delivery)** 하는 활동들
- SW는 첫 버전 후에도 고정되지 않는다: 버그 발견·수정, 요구사항 변경(정부 규제, 표준화), 하드웨어 변경, 사용자 기대 변화(경쟁 제품 때문에 기능 추가), 더 나은 문서 요구
- **좋은 설계는 변화가 불가피함을 인정하고 처음부터 대비한다**
- **Prepare for Change**: 가장 가능성 높은 변화 원인을 예측해 영향을 격리(인터페이스, 파일 형식, 통신 프로토콜), 바뀔 하드웨어 인터페이스 격리, **구성요소 간 의존 줄이기**, 주요 결정의 이유를 설계 문서에 기록

흔한 설계 결함 (Common Design Flaws)

- **Direct modification**: 다른 구성요소의 데이터를 직접 수정 → **캡슐화 위반**
- **Too much responsibility**: 이해하고 쓰기 어려움
- **No responsibility**: 쓸모없음
- **Unused responsibility**: 어떻게 쓰일지 생각하지 않고 설계한 결과
- **Misleading names**: 이름은 짧고 책임을 모호하지 않게 드러내야 함

> 시험 포인트: 2023 빈칸 "(Software maintenance) is the process of changing, modifying, and updating software to keep up with customer needs. (It) is done after the product has launched".

U4 · L04 C++ Data Abstraction.pdf + Handout 2 + 보충 자료

C++ 데이터 추상화: class

교수님이 "이 수업에서 가장 중요한 부분"이라고 한 단원. 데이터 추상화·캡슐화·정보 은닉, 캡슐화의 이점 3가지, public 데이터를 피하는 이유 3가지, 생성자, 복사 생성자, static, const 멤버 함수, inline, friend, 참조, 인자 전달 방식까지. 10/7 수업은 생성자까지 나갔고 10/14 수업은 녹음 전사가 없어. 그 뒤 내용도 기출 단골이라 같이 정리했어.

L4-1 · C++ 데이터 추상화: CLASS

데이터 추상화 · 캡슐화 · 정보 은닉

교안 범위: L04 p.2–4 + Handout 2

§ 세 용어

용어	정의
Data Abstraction	불필요한 세부를 감추는, 적절한 기능을 가진 클래스를 만드는 것
Data Encapsulation	구현이 인터페이스로 **감싸여(encapsulated)** 있어서, 구현에 **합법적으로 접근하는 유일한 방법이 인터페이스**인 것
Information Hiding	구현이 사용자로부터 **숨겨져** 있는 것

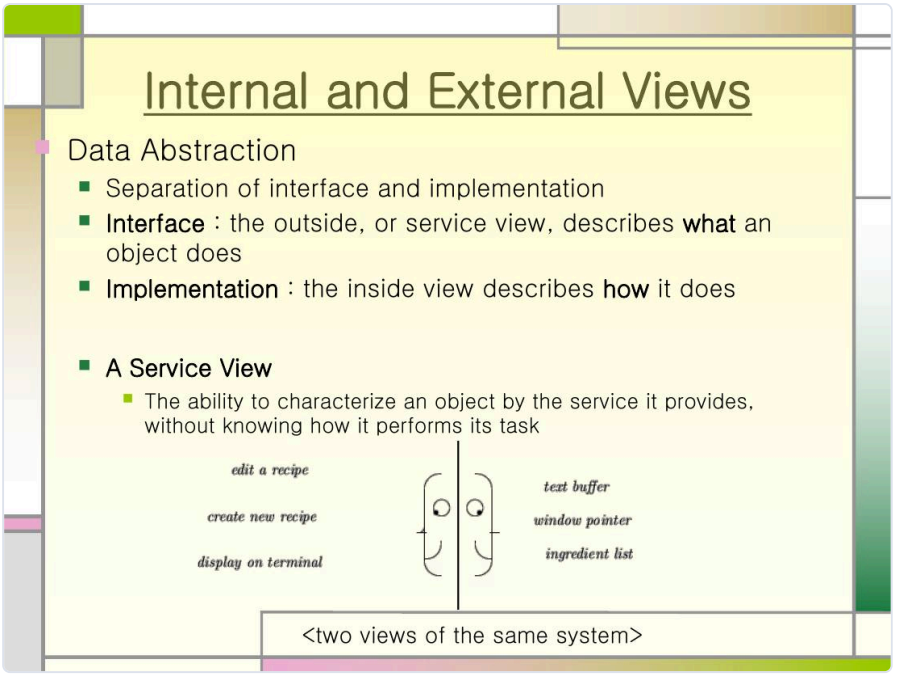
- 세 용어는 **거의 같은 개념**, 강조점만 조금 다름 (추상화 = 핵심만 보여주기, 캡슐화 = 데이터와 함수를 묶어 인터페이스로만 접근, 은닉 = 숨기기)

> 시험 포인트: 교수님이 "취업 면접에서도 나오는 질문, 이 슬라이드가 가장 중요하다"고 함. 모범 답안 = **"Separation of interface and implementation, such that the implementation is hidden from the user and the only way to access it is through the interface."**

§ 내부 관점과 외부 관점

- **데이터 추상화 = 인터페이스와 구현의 분리 (separation of interface and implementation)**
- **Interface**: 바깥, 즉 **서비스 관점(service view)**. 객체가 **무엇을(what)** 하는지 기술
- **Implementation**: 안쪽 관점. **어떻게(how)** 하는지 기술
- **Service view**: 객체가 **어떻게** 일을 수행하는지 몰라도 **제공하는 서비스**로 객체를 특징짓는 능력
- 예) 프로젝터: 버튼(화면 올리기·내리기, 켜기·끄기) = 인터페이스, 안의 회로 = 구현. 사용자는 버튼만 알면 됨

> 시험 포인트: 2019·2016 빈칸 "(Data abstraction) is a technique that relies on the separation of interface and implementation. Interface … describes (what) an object does. Implementation describes (how) it does".



L04 p.3 · Internal and External Views

§ 데이터 캡슐화의 이점 3가지 (Handout 2)

1. **Protection of implementation (safety)**: 클라이언트는 캡슐화된 데이터에 **직접 접근할 수 없다**
2. **Readability (understandability)**: 인터페이스를 쓰는 클라이언트는 인터페이스 함수들이 캡슐화된 정보를 **어떻게 쓰는지 알 필요가 없다**
3. **Implementation independence**: **구현의 변경이 클라이언트가 보는 인터페이스에 영향을 주지 않는다**. 캡슐화된 구현은 구현자가 인터페이스를 건드리지 않고 **자유롭게 바꿀 수 있게** 해준다 (클라이언트가 인터페이스를 쓰는 방식은 구현과 독립)

> 시험 포인트: **최다 출제**. 2025·2022·2017 "List three main benefits ... explain each", 2023 빈칸 "(implementation independence) means (a change) in implementation should not affect (the interface) seen by clients", 2019 "(1) Clients cannot (directly access encapsulated data) (2) Clients of interface does not need to (know how ...)". 세 문장을 영어 그대로 외워.

클래스의 행동과 상태 · 메서드 · 메시지 전달

교안 범위: L04 p.5–8

§ 행동과 상태 · 메서드의 세 부분

- **Class**: C++에서 데이터 추상화의 기본 단위
- **Behavior (method, member function)**: 요청에 응답해서 인스턴스가 수행할 수 있는 행동
- **State (variable)**: 행동을 성공적으로 완수하려고 객체가 유지해야 하는 데이터. 인스턴스 변수(data member, data field)에 저장
- **메서드의 세 부분**
  - **Name**: 메시지와 매칭되어 언제 실행할지 결정
  - **Signature**: **반환 타입과 인자 타입의 조합**. 이름이 같아도 시그니처가 다르면 구별됨
  - **Body**: 메서드가 호출되면 실행되는 코드

§ 메시지 전달 (Message Passing)

메시지와 함수 호출의 차이

- 메시지는 항상 **어떤 객체(receiver)** 에게 주어진다
- 응답으로 하는 행동은 **수신자가 결정** → 수신자가 다르면 같은 메시지에도 다른 행동을 할 수 있다

메시지 전달 문법의 세 부분: `aGame.displayCard(aCard, 42, 27);`

- **message receiver**: `aGame`
- **message selector**: `displayCard`
- **arguments** (선택): `aCard, 42, 27`

public 데이터 피하기 · getter/setter · 불변식과 assert

교안 범위: L04 p.9–12

§ public 영역에 데이터 멤버를 두지 마라

문법적으로는 문제없지만 OOP 관점에서 좋지 않다. 이유 3가지:

1. **Consistency**: 멤버 변수에 접근할 때 **항상 인터페이스(함수)로** 한다는 일관된 정책
2. **Precise control over the accessibility of data members**: no access, **read-only, read-write, write-only**를 정확히 제어
3. **Functional abstraction**: 나중에 데이터 멤버를 **계산(computation)으로 바꿀 수 있다** (예: 접근 횟수 세기 같은 로직 추가)

```
class AccessLevels {
public:
  int  getReadOnly() const     { return readOnly; }
  void setReadWrite(int value) { readWrite = value; }
  int  getReadWrite() const    { return readWrite; }
  void setWriteOnly(int value) { writeOnly = value; }
private:
  int noAccess;   // 접근 불가
  int readOnly;
  int readWrite;
  int writeOnly;
};
```

시험 포인트: 2024·2022 빈칸 "(i) (Consistency) (ii) precise (control) over the (accessibility) of data members (iii) functional (abstraction): you can later (replace) the data member with (a computation)". 2024·2022 서술 "every member private이면? → no interface, clients cannot use the object / every member public이면? → no encapsulation, clients can directly access and corrupt data".

§ Accessor(getter)와 Mutator(setter)

- **Accessor (getter)**: 내부 데이터 값을 **그냥 반환**하는 메서드. 쓰는 이유: 데이터를 **read-only**로, 데이터에 접근할 수 있다는 걸 **더 잘 문서화**(인터페이스만 봐도 알 수 있음), 나중에 접근 방식을 바꾸기 쉬움 (예: 접근 횟수 세기)
- **Mutator (setter)**: 객체의 변수를 **바꾸는** 메서드. getter보다 **덜 흔함** (쓰기는 위험하므로 가능하면 최소화)

§ 클래스 불변식과 assert

- **Class invariant**: 객체 안에서 **항상 참이어야 하는 조건**. 모든 멤버 함수 코드가 이 조건을 검사하도록 조건을 추가
- C/C++의 **assert 매크로** `assert(e)` (`#include <assert.h>` 또는 `<cassert>`)
  - **e가 참이면 그냥 계속**
  - **e가 거짓이면 프로그램이 assertion 실패 에러 메시지와 함께 멈춤**
- **목적: 더 신뢰할 수 있는(reliable) 프로그램** → 버그를 빨리 찾고 고치기
- 교수님: "가능한 한 assert를 많이 넣어라"
- 함정: `assert(x = 3)` 은 대입이라 항상 참 (3은 0이 아니니까). 비교는 `assert(x == 3)`

시험 포인트: 2025 서술 "(i) What does assert(e) do? (ii) main purpose? (iii)...", 2022 빈칸 "(An assertion) contains a boolean expression that you believe will be true when it executes... writing (assertions) helps find (bugs)".

PlayingCard 예제 · 가시성 · 생성자 · 기본 생성자

교안 범위: L04 p.13–16

§ PlayingCard 클래스와 가시성

```
class PlayingCard {
public:
  enum Suits {Spade, Diamond, Club, Heart};
  enum Colors {Red, Black};
  PlayingCard ();
  PlayingCard (Suits is, int ir)                 // constructor
    { suitValue = is; rankValue = ir; faceUp = true; }
  ~PlayingCard () {}                             // destructor
  Suits suit () const { return suitValue; }      // getter
  int rank () const;                             // getter
  void setFaceUp (bool up) { faceUp = up; }      // setter
  Colors color ();
private:
  Suits suitValue;
  int rankValue;
  bool faceUp;
};
```

- color는 데이터로 저장하지 않음 → suit에서 **계산** (Diamond·Heart면 Red)
- **Visibility modifier**: 인터페이스와 구현 관점을 구분
  - **public**: 외부(인터페이스, 서비스) 관점. 누구나(모든 클라이언트) 보고 조작
  - **private**: 내부(구현) 관점. **클래스 안에서만** 접근 (그 클래스의 멤버 함수만)
  - **protected**: 상속에서 다룸 (상속이 없으면 private과 같음)
- 설계 순서: 필요한 **인터페이스(public)** 를 먼저 정하고 → 그걸 구현하는 데 필요한 함수·변수를 **private**에

§ 생성자 (Constructor)

- 새로 만든 객체의 **데이터 멤버를 초기화**하는 메서드
- 객체가 **생성될 때 자동으로(implicitly) 호출** (100%)
- **반환 타입·반환 값이 없다**
- **클래스와 이름이 같다**
- 여러 개 가능 (매개변수가 다르면)
- 초기화 자체가 자동인 건 아님 → 생성자 안에 초기화 코드를 써야 함. 안 하면 쓰레기 값 → 위험

멤버 초기화자 (member initializer)

```
PlayingCard (Suits is, int ir) : suitValue(is), rankValue(ir) { faceUp = true; }
```

> 시험 포인트: 2019 빈칸 "(2) automatically called when (an object is created) (3) should always have the same (name) as the class (4) does not have (return) type", 2017 "automatically called when (an object) is (created)... the name is the same as (the class)... used to (initialize) data members".

§ 기본 생성자와 소멸자

- 클래스에 **생성자를 하나도 선언하지 않으면 컴파일러가 기본 생성자(default constructor)를 만든다**
  - 모양: `TIntStack::TIntStack() { /* Empty */ }` (매개변수 없고 몸체가 빔)
  - **생성자를 하나라도 만들면** 기본 생성자는 **자동 생성되지 않는다**
- **소멸자 (destructor)**: 객체가 메모리에서 삭제될 때 자동 호출. `~클래스이름()`, 하나뿐. 안 만들면 컴파일러가 빈 소멸자를 만듦

> 시험 포인트: 2025·2024 빈칸 "if we do not declare any constructor in a class, (the compiler) generates (a default constructor)".

복사 생성자 · 얕은/깊은 복사

교안 범위: L04 p.17–18

§ 복사 생성자가 호출되는 때

새 객체가 같은 타입의 기존 객체로부터 만들어질 때 호출된다.

```
Person q("Mickey");   // 일반 생성자로 q를 만듦
Person r(p);          // 복사 생성자로 r을 만듦
Person p = q;         // 선언에서 초기화 → 복사 생성자
f(p);                 // 값 매개변수 초기화 → 복사 생성자
p = q;                // 대입 연산자! 생성자도 복사 생성자도 아님 → p.operator=(q)
```

- 선언: `Person(const Person& p) { … }`
- 대입 연산자: `Person& operator=(const Person& p) { … }`

> 시험 포인트: 2017 "(a) Copy constructor is automatically called when (a new object is created from an existing object of the same type). (b) 세 경우의 예" → `Person r(p);` / `Person p = q;` / `f(p);`. 그리고 `p = q;`는 아니라는 것.

§ 얕은 복사와 깊은 복사

- 복사 생성자를 안 만들면 컴파일러가 **기본 복사 생성자, 기본 대입 연산자, 기본 소멸자**를 만든다 → **얕은 복사(shallow copy)**: 멤버 값을 그대로 복사 (**포인터면 주소만 복사**)
- 얕은 복사로 괜찮으면 복사 생성자를 쓰지 마라
- 동적 할당한 메모리를 가리키는 포인터 멤버가 있으면 **깊은 복사(deep copy)** 가 필요: **새 메모리를 new로 할당하고 내용을 복사**
- 얕은 복사의 문제: 두 객체가 **같은 배열을 공유** → 한쪽을 바꾸면 다른 쪽도 바뀜, 둘 다 소멸될 때 **delete[]가 같은 메모리에 두 번** (double free, dangling pointer)

§ 복사 생성자가 필요하면 소멸자와 operator=도

- **복사 생성자가 필요하면 소멸자와 operator=도 필요하다** (Why? 셋 다 같은 이유, 즉 동적 할당 자원을 소유하기 때문: 복사 때 깊은 복사, 소멸 때 delete[], 대입 때 기존 메모리 해제 후 깊은 복사)
- 예) Project 2의 `inf_int`: 자릿수를 동적 할당한 포인터로 가짐 → 깊은 복사를 하는 복사 생성자 필요. 안 쓰면 얕은 복사로 배열 공유, delete[] 두 번

> 시험 포인트: 2016 "Why do we generally need to write a copy constructor in class inf_int? What happens if we don't?", 2017 TIntStack "(4) What problems occur if the copy constructor is not provided?".

객체 생성과 메모리 문제

교안 범위: L04 p.19

§ 객체 생성과 해제

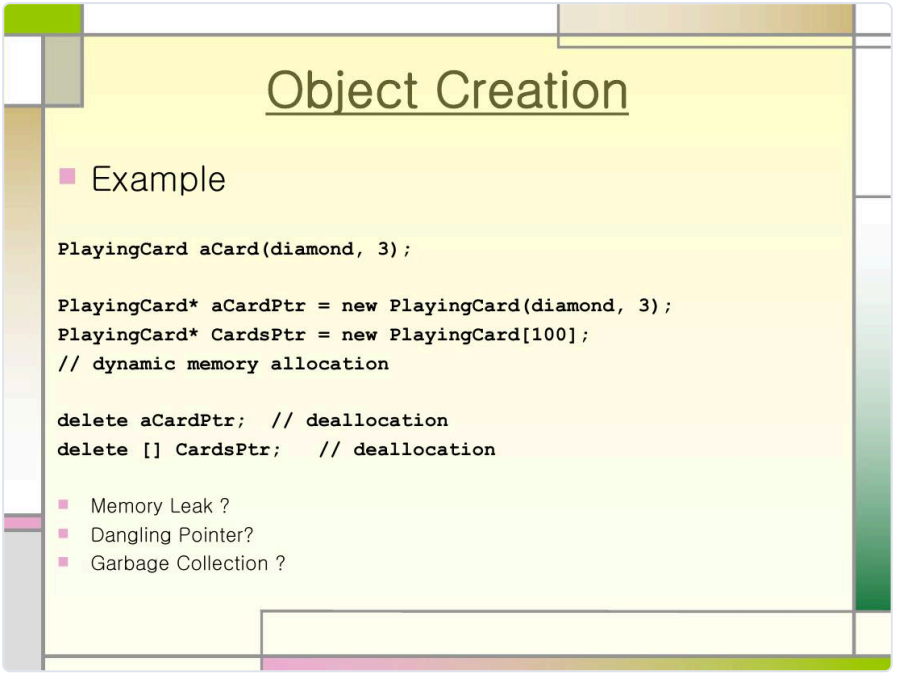
```
PlayingCard aCard(diamond, 3);                          // 정적 (지역 객체)
PlayingCard* aCardPtr = new PlayingCard(diamond, 3);    // 동적
PlayingCard* CardsPtr = new PlayingCard[100];           // 동적 배열 (기본 생성자 100번)
delete aCardPtr;      // 해제
delete [] CardsPtr;   // 배열 해제
```

- 지역 객체는 **만든 역순**으로 소멸, `delete[]`는 배열 원소를 **뒤에서부터** 소멸

§ 메모리 누수 · 댕글링 포인터 · GC

- **Memory leak**: 동적 할당한 메모리를 **해제하지 않아서** 다시 쓸 수 없게 되는 것 (가리키는 포인터를 잃어버림)
- **Dangling pointer**: **이미 해제된 메모리를 가리키는 포인터**. 접근하면 정의되지 않은 동작(undefined behavior) → 충돌, 다른 데이터가 쓰는 메모리를 망가뜨림
- **Garbage collection**: 안 쓰는 메모리를 자동으로 회수 (Java). C++에는 없음

> 시험 포인트: 2025 서술 "(i) What is a dangling pointer? (ii) Why dangerous?", 2024 빈칸 "A pointer pointing to a memory location that has been deallocated is called a (dangling) pointer".



L04 p.19 · Object Creation (memory leak, dangling)

static · const 멤버 함수 · inline · 정의와 구현 분리

교안 범위: L04 p.20–25 + 보충 자료

§ static 멤버

- **static**: 모든 인스턴스가 **같은 값을 공유. 클래스당 하나 (one per class)**
- static 멤버는 **클래스의 속성(property of the class)** 을 다룰 때 쓴다
- **static 멤버 변수**: 클래스당 하나, **객체가 하나도 없어도 접근 가능**
- **static 멤버 함수**: 객체 없이 `클래스이름::함수()` 로 호출 가능, **non-static 멤버 변수·함수에는 접근할 수 없다**
- 클래스 밖에서 정의·초기화: `int File::open_files = 0;`
- `static const int Spade = 1;` (const = 다시 대입하지 않음)

```
class File {
  static int open_files;
public:
  File(const String& filename, const String& modes);
  File(const File&);
  ~File();
  static int existing() { return open_files; }   // 존재하는 File 객체 수
};
int File::open_files = 0;
File::File(const String& filename, const String& modes) { ++open_files; }
File::~File() { --open_files; }
```

시험 포인트: 2025 빈칸 "static members are used to represent the properties of (the class)... can be accessed even if there is no (created object)", 2024 "no matter how many (objects) are created, only one copy", 2022 서술 File 예제 "(3) What problem occurs without static?" → 객체마다 자기 open_files를 가져서 각자 1이 되고, 전체 File 객체 수를 셀 수 없다.

§ const 멤버 함수

- **호출된 객체의 데이터 멤버를 바꾸지 않는** 멤버 함수 (읽기 전용 함수)
- 이 제한은 **컴파일러가 강제(enforced by the compiler)** 한다
- 이점: 클라이언트에게 **해가 없다는 것(benign nature)을 보장** → 안전성, 가독성
- 예: `int PlayingCard::rank () const { return rankValue; }`

시험 포인트: 2025·2024·2023·2022 **4년 연속** 빈칸 "(const) member function does not (modify) any data members of the object on which it is called. This restriction is enforced by (the compiler)".

§ inline 함수

(슬라이드는 질문만 있고 수업에서 설명하는 부분)

- **inline 함수**: 컴파일러가 **함수 호출을 함수 몸체로 바꿔 넣는(호출 위치에 코드를 펼치는)** 함수. 클래스 정의 안에서 몸체를 쓴 멤버 함수는 암묵적으로 inline
- 이득을 보는 상황: ① 몸체가 **작을 때** (getter/setter) ② **아주 자주 호출**될 때 (루프 안)
- 주 이점: **실행이 빠르다** — 함수 호출 오버헤드(인자 전달, 점프, 스택 프레임, 반환)가 없어지니까
- 단점: **코드 크기 증가 (code bloat)**, 특히 함수가 크거나 여러 곳에서 호출될 때

시험 포인트: 2022 서술 4문항 (정의, 이득 상황 2개, 주 이점과 이유, 단점).

§ 메서드 순서 · 정의와 구현 분리

메서드 나열 지침: 중요한 것 먼저, 생성자 먼저, **public을 private보다 먼저**, 긴 목록은 그룹으로, 찾기 쉽게 알파벳 순

클래스 밖 정의

```
PlayingCard::Colors PlayingCard::color ( ) {
  if ((suit == Diamond) || (suit == Heart)) return Red;
  return Black;
}
```

- `클래스이름::` 을 붙인다. 반환 타입도 클래스 안의 타입이면 `PlayingCard::Colors` 처럼
- 생성자·소멸자는 반환 타입 없이 `TIntStack::TIntStack(...)`

friend · 참조

교안 범위: L04 p.26–31 + ref.cpp

§ friend

- 클래스의 **friend 함수는 그 클래스의 non-public 멤버에 접근할 권리**가 있다
- ClassTwo의 모든 멤버 함수를 ClassOne의 friend로: ClassOne 정의 안에 `friend class ClassTwo;`

```
class Count {
  friend void setX(Count&, int);   // friend 선언 (public/private 어디든 가능)
public:
  Count() : x(0) { }
private:
  int x;
};
void setX(Count &c, int val) { c.x = val; }   // 비멤버지만 private x에 접근 OK
```

- **멤버 함수**는 같은 클래스의 다른 객체의 private에도 접근 가능. 막히는 건 **비멤버 함수**뿐 → 고치려면 friend 선언 한 줄

> 시험 포인트: 2023·2022 "code contains one compile error... (a) Which line? (b) Why wrong? (c) Insert one line to fix" → 비멤버 함수가 private에 접근 → `friend 반환형 함수이름(인자);` 를 클래스 안에 추가.

§ 참조 (Reference)

- **참조 = 객체의 별명 (alias for an object)**
- 주로 **참조에 의한 전달 (pass-by-reference)**: 큰 인자를 값으로 전달하는 실행 시간 비용을 피함
- 매개변수, 반환 값, 변수 모두 참조로 정의 가능

```
double square (double &x) { return x *= x; }
double foo = 10.0;
square(foo);   cout << foo;   // 100 (원본이 바뀜)
```

- 함수가 참조를 반환 → lvalue: `v[3] = 100;` (= `v.operator[](3) = 100;`)

ref.cpp

```
int x=5;  int* y;  int& z=x;
y=&x;  int w=*y;  x=9;
z=*y*x;  x++;  z=x&*y;
cout << x << " " << *y << " " << z << " " << w << endl;   // 82 82 82 5
```

(z는 x의 별명, y는 x를 가리킴 → x=9 → z=9*9=81 → x++ = 82 → z = 82 & 82 = 82, w는 처음 값 5)

§ 참조 vs 포인터

- 참조는 **const 포인터와 비슷하게 구현**된다
- 차이: **참조는 반드시 실제 객체를 가리켜야** 하지만, 포인터는 객체가 아닌 것도 가리킬 수 있다 (예: 0, NULL)
- 일반적으로 **참조가 더 안전하고, 덜 모호하고, 더 제한적**

```
void printDouble(const double& rd) { cout << rd; }          // 검사 필요 없음
void printDouble(const double *pd) { if (pd) cout << *pd; } // NULL 검사 필요
```

인자 전달 방식 · 메모리 안전 · 지역 참조 반환

교안 범위: L04 p.32–37

§ 인자 전달 모드 (클라이언트 관점)

```
1. void X::f(T arg)                // pass by value
2. void X::f(const T arg)          // pass by value
3. void X::f(T& arg)               // pass by reference
4. void X::f(const T& arg)         // pass by reference
5. void X::f(T* argp)              // pass by pointer
6. void X::f(const T* argp)        // pass by pointer
7. void X::f(T* const argp)        // pointer is constant
8. void X::f(const T* const argp)
```

- **읽기 전용 매개변수에 const 없는 참조 전달을 절대 쓰지 마라**
- `T&` vs `T`: ① **복사를 피함** (복사 생성자 호출 없음 → 큰 객체에 효율적) ② 함수가 **원본을 바꿀 수 있음**
- `const T&` vs `T&`: 복사는 피하면서 **원본을 못 바꾸게** 보장 (안전)

> 시험 포인트: 2022 기말 서술 "(1) Two purposes of '&' in (b) vs (a)? (2) Purpose of const in (c) vs (b)?".

§ 올바른 모드 고르기

- **큰 객체를 값으로 넘기지 마라** — 왜? 객체 전체가 매개변수로 **복사**되므로 (복사 생성자 호출, 메모리 할당과 복사, 끝나면 소멸자) 오버헤드가 큼. 참조는 **주소만** 넘기므로 오버헤드가 아주 작다
- 기본 타입이나 작은 객체는 **값으로** (더 안전)
- 가능한 곳마다 **const** 를 인자와 함수에 → 더 견고한 프로그램
- 객체를 넘길 때 포인터를 넘기지 마라 `void f(T* object);` — 포인터는 NULL일 수 있지만 null 참조는 없다, 참조 전달은 객체의 소유권을 넘기지 않는다

> 시험 포인트: 2017 서술 "(a) Why does high overhead occur when passing large objects by value? (b) Why is the overhead very small with call-by-reference?".

§ 메모리 안전한 클래스 · 클라이언트의 책임

Writing memory safe classes

- 객체 복사를 피해라 (비쌈), 새 객체 생성을 피하고 재사용
- const 참조 매개변수, const 멤버 함수를 써라
- 데이터 멤버로는 참조 대신 포인터를, 기본 생성자에서 저장소 할당을 미루기, 가능하면 참조 계수(reference counting)
- 초기에 트릭 쓰지 말기: **안전 먼저**(메모리 누수 없게), 최적화는 나중에

Client's responsibilities (요약): 클래스의 목적을 이해, 구현자가 기대하는 것(계약)을 이해, 멤버 함수 특히 const 멤버 함수에 주의, 인자를 이해, 반환된 포인터·참조에 대한 책임 이해, 문서에 없는 세부에 의존하지 말기, const 포인터·참조를 받는 함수를 선호

§ 지역 객체의 참조를 반환하지 마라

```
int  f () { int i = 2; return i; }    // OK (값 복사)
int* g () { int j = 2; return &j; }   // 경고: 지역 변수 주소 반환
int& h () { int k = 2; return k; }    // 경고: 지역 변수 참조 반환
```

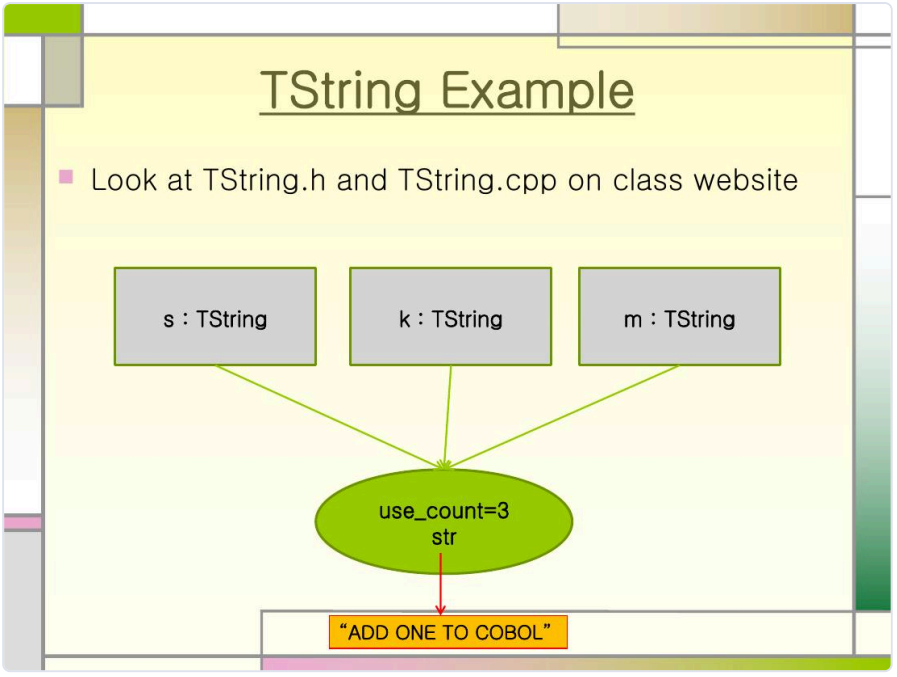
- 함수가 반환되면 지역 변수는 사라진다 → 그 주소·참조는 **댕글링** → 정의되지 않은 동작

Copy-On-Write · TIntStack

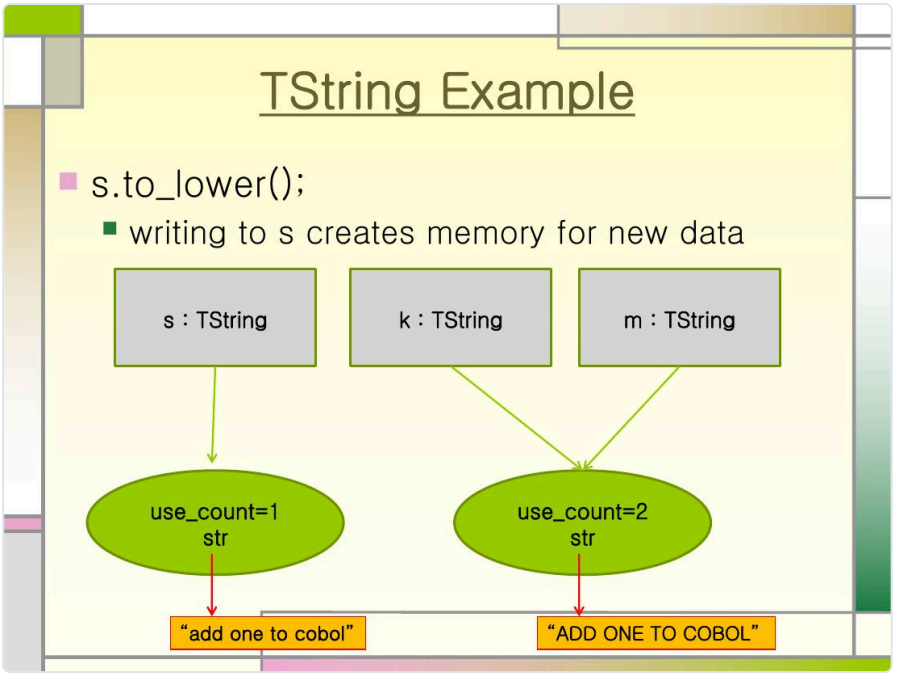
교안 범위: L04 p.38–40 + 기출

§ Copy-On-Write (TString)

- 객체 생성·소멸은 비싸다. 큰 객체의 복사 생성자가 자주 불리면 동적 메모리 복사·삭제에 많은 시간
- **Copy-on-write**: **쓸 때만** 새 메모리를 할당. 그 전엔 원본과 새 객체가 데이터를 **공유**
- **참조 계수(use_count)** 를 유지
- 예) s, k, m 세 TString이 "ADD ONE TO COBOL" 하나를 공유 (use_count=3) → `s.to_lower();` 로 s에 쓰는 순간 s만 새 메모리 ("add one to cobol", use_count=1), k와 m은 계속 공유 (use_count=2)



L04 p.39 · TString: 세 객체가 데이터 공유 (use_count=3)



L04 p.40 · s.to_lower() 뒤: s만 새 메모리

§ TIntStack (깊은 복사 구현)

동적 배열로 만든 정수 스택. 기출에서 생성자·복사 생성자·소멸자·Push를 직접 쓰게 함 (2022, 2017).

```
class TIntStack {
public:
  TIntStack(unsigned int stackSize = DEFAULT_SIZE);
  TIntStack(const TIntStack& that);
  ~TIntStack();
  bool Push(int thisValue);           // 꽉 찼으면 false
private:
  int* _sp;              // 생성자에서 동적 할당
  unsigned int _count;   // 원소 수
  unsigned int _size;    // 용량
};
TIntStack::TIntStack(unsigned int stackSize) { _size = stackSize; _count = 0; _sp = new
int[_size]; }
TIntStack::TIntStack(const TIntStack& that) {          // 깊은 복사
  _size = that._size; _count = that._count;
  _sp = new int[_size];
  for (unsigned int i = 0; i < _count; i++) _sp[i] = that._sp[i];
}
TIntStack::~TIntStack() { delete[] _sp; }
bool TIntStack::Push(int thisValue) {
  if (_count ⩾ _size) return false;
  _sp[_count++] = thisValue;
  return true;
}
```

- 클래스 밖 정의: **반환 타입 없이**(생성자·소멸자), `TIntStack::` 붙이고, 다른 객체는 `that._size`처럼 `.`
- `_sp = that._sp;` 만 하면 얕은 복사 → 감점
- 스택 = **LIFO** (last in, first out): 1, 3, 4 순서로 push하면 pop은 4, 3, 1

U5 · Operator Overloading (Proj2 참고).pdf + Project2/src/complex.cpp

연산자 오버로딩 (Project 2 참고)

Project 2(inf_int)용 자료. 프로젝트 마감(11/1)이 시험 뒤라 코딩 문제로 나올 가능성은 낮지만, 멤버 함수 방식과 friend(비멤버) 방식의 차이는 개념 문제로 나올 수 있어.

연산자 함수: 멤버 방식 vs 비멤버(friend) 방식

교안 범위: Operator Overloading p.1–3

§ 연산자 함수와 두 가지 방식

- **연산자 함수 (operator function)**: 연산자(`+ - * / [] << >>` …)를 **함수로 다시 정의**
- **멤버 함수 방식**
  - `this`로 인자 하나를 암묵적으로 받음 → 이항 연산자의 **왼쪽 피연산자**
  - **가장 왼쪽 객체가 연산자와 같은 클래스**여야 함
  - `c = a + b;` → `c = a.operator+(b);`
- **비멤버 함수 방식**
  - **두 피연산자 모두 매개변수**로 받음
  - 왼쪽 객체가 **다른 클래스**여도 됨 (예: `cout << obj`)
  - private/protected 데이터에 접근하려면 **friend**여야 함
  - `c = a + b;` → `c = operator+(a, b);`

> 시험 포인트: "멤버 방식에서 왼쪽 피연산자는 무엇으로 받나 (this)", "비멤버 방식이 private에 접근하려면 (friend)".

§ Complex 예제

비멤버(friend) 방식

```
class Complex {
private:
  double re, im;
public:
  Complex(double r, double i) : re(r), im(i) {}
  friend Complex operator+(Complex&, Complex&);
  friend Complex operator-(Complex&, Complex&);
  friend Complex operator*(Complex&, Complex&);
  void Display() { ... }
};
Complex operator+(Complex& i, Complex& other) {
  return Complex(i.re + other.re, i.im + other.im);
}
Complex operator*(Complex& i, Complex& other) {
  return Complex(i.re*other.re - i.im*other.im, i.re*other.im + i.im*other.re);
}
```

멤버 방식

```
class Complex {
  ...
  Complex operator+(Complex& other);
};
Complex Complex::operator+(Complex& other) {
  return Complex(re + other.re, im + other.im);
}
```

- a = 1.2+3.4i, b = 5.6+7.8i → a+b = 6.8+11.2i, a−b = −4.4−4.4i, a×b = −19.8+28.4i (두 방식 결과 같음)

- Project 2의 `inf_int`도 같은 방식으로 `+ - * ( )`를 지원하는 무한 정수 계산기를 만든다. `inf_int`는 자릿수를 동적 할당하므로 **복사 생성자·소멸자·operator=** 가 필요 (L4-5)